Développement mobile - Projet 2026

Objectif

L'objectif de ce projet est de créer une **application mobile** développée en **ReactJS** qui permet de simuler un **jeu de combat de cartes simplifié** (inspiré de jeux comme *Magic* ou *Hearthstone*).

L'application doit remplir les objectifs suivants :

- **API Externe (Lecture) :** Utiliser une **API web externe** de votre choix (soumise à validation pour alimenter le contenu des cartes (noms, images, descriptions, statistiques).
- **Firebase / Authentification (Écriture) :** Utiliser **Firebase** pour gérer la persistance des données utilisateur et l'**Authentification Google** pour l'accès.
- **Jeu Simplifié :** Développer une logique de jeu simple, permettant au minimum à l'utilisateur de :
    - S'authentifier via Google.
    - **Consulter** sa collection de cartes (données lues depuis l'API externe).
    - **Construire** son deck de combat (données stockées/écrites via Firebase).
    - **Simuler un combat Joueur contre Joueur (PvP)** en temps réel, où **Firebase gère l'état et la synchronisation** de la partie entre les deux adversaires.

Exemple d'API externes :

- *PokéAPI*
- *Star Wars API*
- *Ice & Fire API*

L'idée est de modéliser le combat dans Firebase (par exemple, un document `parties/{partieId}`) pour qu'il contienne l'état des deux joueurs (Joueur A et Joueur B).

Élément	Détails de la mise en œuvre Firebase
Démarrage	Un joueur initie une partie et **crée un document de partie** dans Firebase. Un autre joueur peut "rejoindre" cette partie.
Structure de la Carte	Chaque carte a deux statistiques : **Attaque (ATK)** et **Défense (DEF)**. **Ces valeurs doivent être générées aléatoirement** (par exemple entre 1 et 10) et **stockées dans Firebase** en complément des données de l'API

	externe. Ces stats sont générées une seule fois pour le jeu puis **persistées via Firebase**. Elle sont communes à tous les utilisateurs.
Statut de Jeu	Le document Firebase doit stocker le statut de la partie : le **joueur actif** (celui dont c'est le tour), les **Points de Vie (PV) des deux joueurs**, les **cartes jouées** sur le terrain et le **deck restant de chaque joueur**.

Règle du jeu

Chaque joueur doit construire un deck de **10 cartes différentes**. Chaque carte a deux statistiques : **Attaque (ATK)** et **Défense (DEF)**. Chaque joueur commence la partie avec **5 Points de Vie (PV)**. Le premier joueur est choisi aléatoirement (à l'initialisation de la partie).

1. Mise en Place du Terrain

- Chaque joueur pioche 3 cartes aléatoires dans son deck. **Ces 3 cartes sont immédiatement placées sur le terrain**.
- L'état de ces cartes (ATK/DEF) est stocké dans le document de partie pour chacun des joueurs comme les cartes présentes sur le terrain.

2. Tour du Joueur Actif (Attaque)

- **Réalimentation du terrain :** En début de tour, si le terrain du joueur actif contient moins de 3 cartes, **il pioche aléatoirement des cartes de son deck pour en avoir 3 sur le terrain.** Si le deck est vide, il ne peut plus réalimenter le terrain.
- **Attaque obligatoire :** Le joueur choisit obligatoirement **une carte de son terrain** pour attaquer.

3. Tour de l'Adversaire (Défense)

- **Défense réactive :** L'adversaire *peut* désigner une de ses cartes sur le terrain pour bloquer l'attaque (si possible), OU il encaisse les dégâts sur ses PV.

4. Résolution & Défausse

- **Clash de carte :** L'ATK de l'attaquant est comparée à la DEF du défenseur. Lorsqu'une carte attaque une carte adverse, les deux cartes sont **toujours défaussées** (retirées du jeu) après le combat, quelle que soit l'issue. Si ATK > DEF, le défenseur perd **1 PV**.
- **Attaque directe sur PV :** Si le défenseur laisse passer l'attaque, alors le défenseur perd **1 PV**, et la carte attaquante n'est pas défaussée.
- **Objectif :** Le but du jeu est de réduire à néant les PV de l'adversaire.

5. Fin de Jeu

- Un joueur atteint 0 PV (perdant).

- Les deux joueurs n'ont plus de carte sur le terrain (match nul).
- L'application met à jour le document de la partie avec le résultat (Victoire/Défaite), stocké dans Firebase.

Critères d'évaluation

Le travail se fera en binôme issu du même groupe. Les exigences techniques doivent être strictement respectées et comptent pour un grande partie de la note finale. Le travail est à livrer en continue sur GitHub grâce à cette invitation classroom pour obtenir un dépôt partagé privé.

- Respect du sujet.
- Richesse fonctionnelle.
- Respect des exigences techniques.
- Qualité de l'ergonomie/UX (présentation, possibilités de navigation, etc).
- Robustesse (comportement en cas de saisie erronée, d'erreur réseau, etc).
- Respect des grands principes de conception logicielle (séparation des responsabilités, nommage et formatage du code, etc).
- Qualité du README (fond et forme).

Il est également requis de soumettre votre jeu à des tests utilisateurs qualitatifs menés auprès de vos camarades (entretiens - toutes promotions confondues). Un fichier `TESTS_UTILISATEURS.md` documentera les conclusions et les évolutions effectuées suite à ces tests (sans entrer dans le détail de la sélection des testeurs, etc). Les tests sur maquette papier avant implémentation sont bien évidemment autorisés.

La date limite de dépôt des livrables est le **12 avril 2026 23h59**.

Un oral technique le **24 avril 2026 (Attention les heures ADE peuvent être ajustées)**.

Exigences techniques

Code	Description
Tech_01	L'application est développée en ReactJS et basée sur le langage JavaScript.
Tech_02	Tous les fichiers sources sont formatés à l'aide de l'outil Prettier.
Tech_03	La structure de l'application distingue : - les service pour les accès externes (**services**/) - les composants élémentaires (**components**/) - les pages principales de l'application (**pages**/) - les fichiers annexes non javascript (**assets/**)
Tech_04	La granularité des composants est la plus fine possible (autrement dit : un composant trop "gros" doit être découpé en sous-composants plus

	élémentaires).
Tech_05	Le projet doit être documenté dans un fichier README.md situé à la racine du dépôt git. Le fichier doit être rédigé au format Markdown. La documentation README.md doit contenir dans l'ordre : ● une présentation générale du projet, rédigé en vos propre termes ; ● la liste complète des fonctionnalités ; ● l'url du site déployé ; ● la procédure d'installation en local. *Corollaire : Pas de rapport !*
Tech_06	Le gestionnaire de version **git** doit être convenablement utilisé tout au long du projet : ● la progression du projet doit être apparente dans l'historique des commit ; ● les messages des commit doivent indiquer l'exigence concernée (une seule en générale) ; ● tous les membres de l'équipe doivent "commiter" ; ● Ne doivent pas être présents les fichiers node_modules (!!).
Tech_07	L'application doit utiliser MaterialUI pour l'interface graphique pour **toutes les structures** (navigation, écrans de connexion/profil, gestion du deck). Pour la **vue de combat**, vous devrez étendre et styliser les composants MaterialUI.
Tech_08	L'application doit être déployée sur Netlify (sera couvert en mini-tp).
Tech_09	L'application doit comporter des tests unitaires couvrant sur la logique du jeu et la constitution du deck, *en bouchonnant les **méthodes des services d'accès à l'API externe et à Firebase**.*